These are the key things the development team noticed:

- ***Velocity was going down again.*** This indicated that they were still being impacted by the split in work and weren't able to adjust.
- ***Velocity for new features dropped sharply.*** This indicated that the work being done by the refactor was severely hurting the team's ability to deliver anything new.
- ***CLOC was going up.*** The team was changing a lot more code.
- ***Bugs and recidivism were rising.*** The overall product was suffering for the combination refactor and feature work.

According to this, the refactor was causing significant churn in the codebase and apparently leaving a host of new bugs in its wake. New feature development velocity had hit the floor because that work was being done on top of a shaky foundation.

The team needed to adjust to these findings. Team members got together and started reviewing the list of bugs to determine root cause and validate or disprove their hypotheses. They discovered that as the team dug into the refactor, changes they made in one part of the codebase were causing other parts to break. They had a general lack of unit tests across the integrated product that caused them to find bugs after the refactored code was delivered. This led to a Whac-A-Mole situation, where fixing one issue ended up creating at least one more to chase after.

2.4.1. Solving the problems

The team knew they had to make the following changes to get their project back on track:

- Ensure that they had enough automated testing across the application to prevent new bugs from popping up after code was refactored. This should help them find tangential problems as they come up and avoid the Whac-A-Mole effect.
- Stop building features for at least a sprint in order to either